ISLAMIC UNIVERSITY OF TECHNOLOGY

SWE 4301

LECTURE 11

Topic: Dependency Injection

February 3, 2026

PREPARED BY

Maliha Noushin Raida

Lecturer, Department of CSE

Contents

1 Dependency Injection **3**
1.1 Critical Points of Dependency Injection 4
1.2 Inversion of control(IoC) — the concept behind DI(Dependency Injection) . . . 4
1.3 Classes of Dependency Injection 5
1.3.1 Client Class 5
1.3.2 Injector Class 5
1.3.3 Service Class 6
1.4 Types of Dependency Injection 6
1.4.1 Constructor Injection 6
1.4.2 Property-Based Injection (aka setter injection) 7
1.4.3 Method Injection 8

2 Reference **9**

1 Dependency Injection

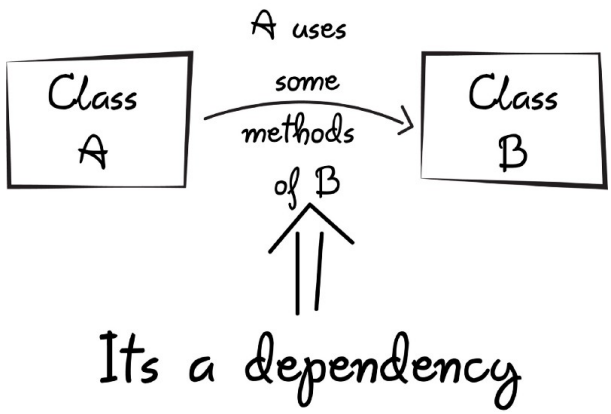

Figure 1: Dependency between classes

Creating robust and maintainable software is constantly pursued in the ever-evolving landscape of Java development. One concept that stands out in achieving these goals is Dependency Injection (DI). This technique enhances code readability and promotes a more flexible and scalable architecture. At its core, Dependency Injection (DI) is a design pattern in Java that addresses the issue of managing dependencies between components in a software system.

In a traditional setup, an object often creates or obtains its dependencies internally, resulting in tightly coupled code that can be challenging to maintain and test. Dependency Injection flips this paradigm by **externalizing and injecting the dependencies into the object**, hence the name "Dependency Injection."

In Java, this often involves **passing the required dependencies as parameters** to a class's constructor or through setters, allowing for a more modular and flexible code structure. Dependency Injection promotes code reusability, testability, and overall system scalability by decoupling components and externalizing dependencies.

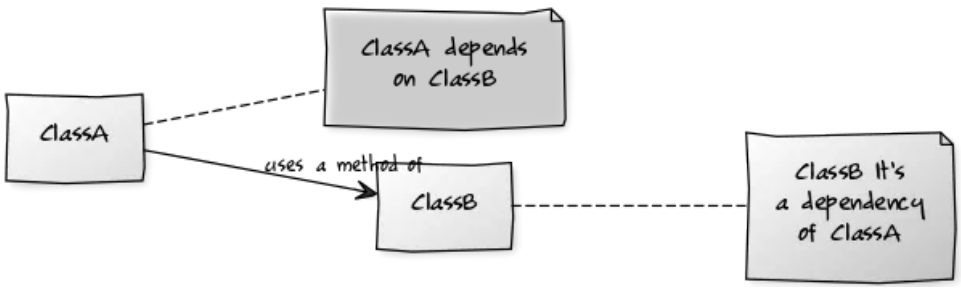

Figure 2: Dependency between classes

```java
class ClassA {
  ClassB classB = new ClassB();
  int tenPercent() {
    return classB.calculate() * 0.1d;
  }
}
```

Listing 1: Code in Java with dependency between classes

When a class ClassA uses any method of another class ClassB, we can say that ClassB is a dependency of ClassA. In this example, ClassA is calculating 10% of the value, and calculating that value, it's reusing the functionality exposed by ClassB. If we needed to change/replace ClassB with ClassC because ClassC has an optimized version of the calculate() method, we need to recompile ClassA because we don't have a way to change that dependency, it's hardcoded inside of ClassA.

1.1 Critical Points of Dependency Injection

- **Decoupling Components**: Dependency Injection reduces the tight coupling between components by ensuring that a class does not create its dependencies internally but receives them from an external source.
- **Modular and Maintainable Code**: With dependencies injected from the outside, each code component becomes more modular and easier to maintain, as changes in one code component do not necessarily affect others.
- **Testability**: DI facilitates unit testing by allowing for easy substituting dependencies with mock objects or alternative implementations. As a result, isolating and testing individual components is simpler.

1.2 Inversion of control(IoC) — the concept behind DI(Dependency Injection)

This states that a class should not configure its dependencies statically but should be configured by some other class from outside.

In the context of Dependency Injection, IoC means that a higher-level component or framework controls the flow and manages the dependencies of lower-level components. In other words, instead of a class controlling the instantiation of its dependencies, the control is inverted, and dependencies come from an external source.

1.3 Classes of Dependency Injection

1.3.1 Client Class

It is the consumer of services or functionalities other classes provide. Its primary purpose is to consume services without being concerned about how they are instantiated or configured. That's why DI relies on externalized dependencies rather than creating them internally. Let's see an example below.

```
public class ProductClient {
  private ProductService productService;
  // Constructor Injection
  public ProductClient(ProductService productService) {
           this.productService = productService;
  }
  public void displayProductDetails() {
           Product product = productService.getProductById(123);
          System.out.println("Product Details: " + product);
  }
}
```

Here, the ProductClient class relies on the ProductService to retrieve product details. The dependency (ProductService) is injected into the ProductClient through constructor injection, promoting loose coupling.

1.3.2 Injector Class

The injector class is responsible for injecting dependencies into client classes. It acts as a bridge between the client and the services it requires. In our example, an injector might look like this:

```
    public class ProductInjector {
    public static void main(String[] args) {
        ProductService productService = new ProductServiceImpl();
        ProductClient productClient = new ProductClient(productService)
   ;
        productClient.displayProductDetails();
    }
}
```

In this example, the ProductInjector creates an instance of ProductService and injects it into the ProductClient. The code sample demonstrates the externalized control of dependencies, a fundamental aspect of Dependency Injection.

1.3.3 Service Class

Service classes encapsulate the business logic and provide functionalities to the client. Let's create a simple implementation for the ProductService:

```
    public interface ProductService {
  Product getProductById(int productId);
}
public class ProductServiceImpl implements ProductService {
  private ProductRepository productRepository;
  // Constructor Injection for Repository
  public ProductServiceImpl() {
      this.productRepository = new ProductRepositoryImpl();
  }
  @Override
  public Product getProductById(int productId) {
      return productRepository.findById(productId);
  }
}
```

1.4 Types of Dependency Injection

1.4.1 Constructor Injection

Constructor injection involves passing dependencies as parameters to a class's constructor. This method ensures that the required dependencies are provided during object creation. Consider the following example:

```
public class ProductServiceClient {
    private final ProductService productService;
    // Constructor Injection
    public ProductServiceClient(ProductService productService) {
        this.productService = productService;
    }
    // Client method using the injected service
    public void displayProductDetails() {
        Product product = productService.getProductById(123);
        System.out.println("Product Details: " + product);
    }
}
```

Constructor injection promotes a clear and explicit declaration of dependencies. It ensures that an object cannot be instantiated without the required dependencies, leading to better

maintainability and reducing the risk of null dependencies. This type of injection is beneficial when dependencies are essential for the proper functioning of the object.

1.4.2 Property-Based Injection (aka setter injection)

Sometimes a class has a dependency that isn't strictly required but is indeed used by the class. An example might be a document class that may or may not have a grammar checker installed. If there is one, great; the class can use it. If there isn't one, great, the class can include a default implementation as a placeholder.

The solution here is property injection (also known as setter injection). You add a property to your class that can be set to a valid instance of the class in question. Since the dependency is a property, you can set it as desired. If the dependency is not wanted or needed, you can leave the property as is.

Your code should act as if the dependency is there, so you should provide a do-nothing default implementation so that the code can still be run with or without a real dependency. Remember, we never want anything to be null, so that default implementation should be valid. If the user of the class wants to provide a working implementation, they can. If not, there is a working default that will allow the containing class to still function.

When to choose this type of injection:
Setter injection provides flexibility, allowing dependencies to be changed or updated after the object is created. It is suitable when a class can function with optional dependencies or when dependencies may change during the object's lifecycle. Setter injection is particularly beneficial for scenarios where the object may be reused with different configurations.

```java
    public class ProductServiceClient {
    private ProductService productService=new defaultProductService();
    // Setter Injection
    public void setProductService(ProductService productService) {
        this.productService = productService;
    }
    // Client method using the injected service
    public void displayProductDetails() {
        Product product = productService.getProductById(123);
        System.out.println("Product Details: " + product);
    }
}
```

1.4.3 Method Injection

What if the dependency that your class needs is going to be different much of the time? What if the dependency is an interface, and you have several implementations that you may want to pass into the class? You could use property injection, but then you'd be setting the property all the time before calling the method that utilized the frequently changing dependency, setting up the possibility of temporal coupling.

Temporal coupling is the idea that the order of execution must occur in a specific way for things to work correctly.

Constructor and property injection are generally used when you have one dependency that isn't going to change often, so they aren't appropriate for use when your dependency may be one of many implementations.

Method injection allows you to inject a dependency right at the point of use so that you can pass any implementation you want without having to worry about storing it for later use. It's often used when you pass in other information that needs special handling. For example:

```
  interface IFoodPreparer {
  prepareFood(aRecipe: Recipe);
}

class Baker implements IFoodPreparer {
  prepareFood(aRecipe: Recipe) {
    console.log('Use baking skills to do the following: ' + aRecipe.
 Text);
  }
}

class ShortOrderCook implements IFoodPreparer {
  prepareFood(aRecipe: Recipe) {
    console.log('Use the grill to do the following: ' + aRecipe.Text)
 ;
  }
}

class Chef implements IFoodPreparer {
  prepareFood(aRecipe: Recipe) {
    'Use well-trained culinary skills to prepare the following: ' +
      aRecipe.Text;
  }
}

```

```
class Restaurant {
  private _name: string;
  constructor(aName: string) {
    this._name = aName;
  }

  set Name(value: string) {
    this._name = value;
  }

  get Name(): string {
    return this._name;
  }

  MakeFood(aRecipe: Recipe, aPreparer: IFoodPreparer) {
    aPreparer.prepareFood(aRecipe);
  }
}
```

Method injection should be used when the dependency could change with every use, or at least when you can't be sure which dependency will be needed at the point of use.

2 Reference

- https://stackify.com/dependency-injection/
- https://builtin.com/articles/dependency-injection
- https://www.mend.io/blog/how-to-use-dependency-injection-in-java-tutorial-with-examples/
- https://www.objc.io/issues/11-android/dependency-injection-in-java/